

Investigating Gradient Flow and Sensitivity in Vision Transformers via Matrix Calculus and Orthogonal Initialization

Nikolaos Mavros

Department of Electrical and Computer Engineering

University of Thessaly, Volos, Greece

ECE452 — Special Topics in Applied Mathematics

nmavros@uth.gr

Source code: github.com/mavroulls/vit-gradient-flow

Abstract—The stability of gradient propagation in deep networks depends on the spectra of their weight matrices. In Vision Transformers (ViTs), repeated multiplication of layerwise Jacobians through self-attention can amplify or attenuate gradients, destabilizing optimization. This work investigates whether *orthogonal initialization* ($W^\top W = I$), which fixes all weight singular values at unity, stabilizes the Jacobian spectrum of a ViT relative to the Xavier/Glorot baseline. We derive the closed-form Jacobians $\nabla_X \text{Attn}(X)$ and $\nabla_W \text{Attn}(X)$ using matrix differential calculus and expose how initialization enters their spectra. We then train two lightweight ViTs (72,074 parameters, two blocks) on MNIST — one per initialization scheme — and measure per-layer gradient norms, per-block Jacobian condition numbers $\kappa(J)$, and input-sensitivity saliency maps across training. Orthogonal initialization yields dramatically better-conditioned attention Jacobians at initialization (up to $46\times$ lower κ in block 1), a gap that block 1 maintains by over an order of magnitude throughout training, and drives faster training-loss descent at every epoch. Block 2’s conditioning becomes more volatile as attention sharpens, consistent with the softmax-Jacobian amplification mechanism identified in our derivation. Final test accuracies are indistinguishable (97.04% vs. 97.13%), but per-class analysis reveals complementary inductive biases: orthogonal favors curved digits, Xavier favors straight ones. Code and pinned environment are available at the linked repository.

I. INTRODUCTION

Gradient stability in deep networks is governed by the spectra of layerwise Jacobians, and a range of weight initialization schemes [1]–[3] were developed to keep these spectra well-behaved. The Transformer [4] and Vision Transformer (ViT) [5] extend this problem to attention-based architectures, where gradient flow is shaped not only by linear projections but also by the softmax nonlinearity at the core of self-attention.

From a matrix-calculus perspective, each layer defines a Jacobian linear operator and backpropagation multiplies these operators along the computational graph [6]. When their singular values deviate systematically from unity, gradients vanish or explode. Orthogonal initialization [3], [7] enforces $W^\top W = I$, fixing all weight singular values at unity in the feedforward setting. Whether this guarantee carries over to ViTs, however, is less clear: the softmax introduces a

non-orthogonal Jacobian factor that interacts with the linear projections.

This work addresses the question both analytically and empirically. We derive the Jacobians $\nabla_X \text{Attn}(X)$ and $\nabla_W \text{Attn}(X)$ of scaled dot-product attention, use the derivation to motivate measurable predictions, and test them on a small ViT trained on MNIST under Xavier and orthogonal initializations. Saliency maps [8] provide a complementary view on how initialization shapes input sensitivity.

Our contributions are:

- A self-contained matrix-calculus derivation of the attention Jacobians with respect to X and to W_Q, W_K, W_V, W_O .
- A reproducible Julia/Flux [9] implementation with manual attention projections to retain full gradient visibility.
- Empirical comparison of Xavier and orthogonal initialization on a 72,074-parameter two-block ViT, reporting training dynamics, per-layer gradient norms, Jacobian condition numbers, and saliency maps.
- Public release of code, pinned environment, and trained weights at <https://github.com/mavroulls/vit-gradient-flow>.

Section II reviews background, Section III derives the attention Jacobians, Section IV describes the experimental setup, Section V reports results, Section ?? interprets them, and Sections VI–VII conclude. ya bit ther

II. BACKGROUND AND RELATED WORK

We review the three areas the remainder of the paper builds on: Vision Transformers, weight initialization, and matrix calculus in backpropagation.

A. Vision Transformers

The Transformer [4] was introduced for sequence-to-sequence tasks in NLP. The Vision Transformer (ViT) [5] adapts it to computer vision by reframing an image as a sequence of tokens.

Given an image of resolution $H \times W$ and patch size P , the image is partitioned into $N = HW/P^2$ non-overlapping

patches. Each patch is flattened and linearly projected into a d -dimensional embedding via a learnable matrix $E \in \mathbb{R}^{d \times P^2}$ (for grayscale inputs). A learnable class token $\mathbf{x}_{\text{cls}} \in \mathbb{R}^d$ is prepended, and learnable positional embeddings $\mathbf{P} \in \mathbb{R}^{d \times (N+1)}$ are added to encode spatial information. The resulting $X^{(0)} \in \mathbb{R}^{d \times (N+1)}$ is the input to the first transformer block.

Each block applies two residual sub-layers with pre-normalization [10]:

$$X' = X^{(\ell-1)} + \text{MHA}(\text{LN}(X^{(\ell-1)})), \quad (1)$$

$$X^{(\ell)} = X' + \text{FFN}(\text{LN}(X')). \quad (2)$$

Multi-head attention applies h parallel scaled dot-product attention operations with per-head projections $W_Q^{(i)}, W_K^{(i)}, W_V^{(i)} \in \mathbb{R}^{d_k \times d}$, $d_k = d/h$, followed by an output projection $W_O \in \mathbb{R}^{d \times d}$. The FFN is a two-layer MLP with GELU activation. After L blocks, the class token is passed through a linear head to produce class logits.

B. Weight Initialization

Xavier/Glorot [1] samples weights uniformly:

$$W_{ij} \sim \mathcal{U}\left[-\sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}, \sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}\right], \quad (3)$$

keeping activation variance approximately constant across layers under a linearity assumption.

Orthogonal [3] initialization constrains

$$W^\top W = I, \quad (4)$$

so that all singular values of W are unity. In practice W is obtained via QR decomposition of a random Gaussian matrix [7]. Under this scheme, linear layers preserve the norm of perturbations, yielding *dynamical isometry*: the end-to-end Jacobian has singular values concentrated near unity at initialization. Whether this property survives the softmax nonlinearity of self-attention is the empirical question of this work.

C. Matrix Calculus and Backpropagation

Matrix calculus describes how matrix-valued functions change under small perturbations [6]. The Jacobian $J(x)$ at x satisfies $df = J(x) dx$. We use the matrix product rule

$$d(AB) = (dA)B + A(dB) \quad (5)$$

together with the chain rule throughout Section III.

Backpropagation through an L -layer network multiplies L layerwise Jacobians:

$$\frac{\partial \mathcal{L}}{\partial h_0} = \frac{\partial \mathcal{L}}{\partial h_L} \prod_{\ell=1}^L \frac{\partial h_\ell}{\partial h_{\ell-1}}. \quad (6)$$

If their singular values systematically deviate from unity, the product decays or grows exponentially with depth — vanishing or exploding gradients. As a scalar summary of conditioning we use

$$\kappa(J) = \sigma_{\max}(J) / \sigma_{\min}(J), \quad (7)$$

which equals one for an isometry and grows with anisotropic amplification.

III. THEORETICAL DERIVATION

We derive the Jacobians of scaled dot-product attention with respect to its input X and its weight matrices, then analyze how their spectra depend on initialization.

A. Setup

Let $X \in \mathbb{R}^{d \times n}$ denote the attention input, with each of the n columns a d -dimensional token embedding. In our ViT, $d = 64$ and $n = N+1 = 17$. We present the single-head case; extension to multi-head is direct. Define

$$Q = W_Q X, \quad K = W_K X, \quad V = W_V X, \quad (8)$$

and the scaled scores and row-wise softmax attention weights

$$S = \frac{1}{\sqrt{d_k}} Q^\top K, \quad A_{ij} = \frac{\exp(S_{ij})}{\sum_k \exp(S_{ik})}. \quad (9)$$

The attention output is

$$\text{Attn}(X) = W_O V A^\top. \quad (10)$$

The transpose on A reflects our column-major token convention: output column i is $\sum_j A_{ij} V_{:,j}$.

B. Differential of the Row-wise Softmax

For $\mathbf{a} = \text{softmax}(\mathbf{z})$ with $\mathbf{z} \in \mathbb{R}^n$,

$$\frac{\partial \mathbf{a}}{\partial \mathbf{z}} = \text{diag}(\mathbf{a}) - \mathbf{a} \mathbf{a}^\top \equiv J_\sigma(\mathbf{a}), \quad (11)$$

a rank- $(n-1)$ symmetric positive semidefinite matrix whose null direction is $\mathbf{1}$ (shift invariance). Because the softmax in (9) is applied row-wise, the Jacobian $\partial \text{vec}(A) / \partial \text{vec}(S)$ is block-diagonal with n blocks of the form $J_\sigma(A_{i,:}^\top)$.

A key observation: when $\mathbf{a}$ is close to one-hot (peaky attention), J_σ has small nonzero eigenvalues and therefore large condition number on its active subspace. Peakiness can therefore hurt Jacobian conditioning independent of the weight spectra.

C. Jacobian with Respect to X

Applying the product rule (5) to (10),

$$d \text{Attn}(X) = W_O [(dV) A^\top + V (dA)^\top]. \quad (12)$$

This splits the propagation of dX into two channels: the *value channel* (first term), where dX changes V but not A ; and the *score channel* (second term), where dX changes A through Q and K .

The value channel gives $W_O (dV) A^\top = W_O W_V (dX) A^\top$, and vectorizing via $\text{vec}(ABC) = (C^\top \otimes A) \text{vec}(B)$:

$$\text{vec}(W_O W_V (dX) A^\top) = (A \otimes W_O W_V) \text{vec}(dX). \quad (13)$$

The score channel, obtained by propagating $dS = \frac{1}{\sqrt{d_k}} [(W_Q dX)^\top K + Q^\top (W_K dX)]$ through the softmax Jacobian and into $(dA)^\top$, yields a more intricate Kronecker expression J_X^{score} whose full form is given in Appendix A. The total input Jacobian is

$$J_X = \underbrace{A \otimes W_O W_V}_{\text{value channel}} + J_X^{\text{score}}. \quad (14)$$

D. Jacobians with Respect to the Weight Matrices

Holding X fixed, the same procedure yields closed-form Jacobians for each weight matrix. Two are particularly clean:

$$\frac{\partial \text{vec}(\text{Attn})}{\partial \text{vec}(W_O)} = A V^\top \otimes I_d, \quad (15)$$

$$\frac{\partial \text{vec}(\text{Attn})}{\partial \text{vec}(W_V)} = A X^\top \otimes W_O. \quad (16)$$

The Jacobians with respect to W_Q and W_K propagate through the softmax and have a more involved form. Their full expressions, in terms of a value-mixing operator and the block-diagonal softmax Jacobian, are given in Appendix A.

For the spectral analysis below, the structural fact that matters is that all four weight Jacobians are Kronecker products in which a weight factor (or weight product) appears explicitly — so initialization enters their spectra directly.

E. Spectral Implications of Orthogonal Initialization

The Kronecker product satisfies $\sigma_i(A \otimes B) = \sigma_p(A) \sigma_q(B)$ [6], so the spectrum of each Jacobian inherits directly from the spectra of the weight matrices entering it. Under orthogonal initialization (4) all weight singular values equal unity; under Xavier initialization (3) they have non-trivial spread, which propagates into the Jacobian spectra.

This motivates two testable predictions, which we examine in Section V:

- 1) At initialization, $\kappa(J_\ell)$ is lower under orthogonal initialization than Xavier in every block.
- 2) The advantage persists during training where attention stays diffuse; in blocks where attention becomes peaked, the softmax Jacobian J_σ dominates conditioning and the weight-spectrum advantage is partially masked.

IV. EXPERIMENTAL SETUP

A. Dataset

We use the MNIST handwritten digit dataset [11]: 60,000 training and 10,000 test grayscale images, each 28×28 pixels, with ten class labels. Pixel values are scaled to $[0, 1]$; no data augmentation is applied.

B. Model Architecture

We implement a lightweight ViT (Table I) with manually-constructed attention projections so that W_Q, W_K, W_V, W_O are directly accessible for gradient diagnostics. Each 28×28 image is partitioned into sixteen 7×7 patches, flattened, and linearly projected to $d = 64$. A learnable class token is prepended and learnable positional embeddings are added, giving an input of shape $\mathbb{R}^{64 \times 17}$. The sequence passes through two pre-norm transformer blocks (Eqs. 1–2) with four attention heads per block and a GELU feed-forward network of hidden dimension 128. Final layer normalization is applied, and the class token is passed to a linear classification head.

TABLE I: ViT architecture. Total: 72,074 trainable parameters.

Hyperparameter	Value
Image size	28×28
Patch size	7×7
Number of patches (N)	16
Embedding dim (d)	64
Attention heads (h)	4
Per-head dim (d_k)	16
Transformer blocks (L)	2
FFN hidden dim	128
Classification head	$64 \rightarrow 10$

C. Initialization Strategies

Two models share this architecture and differ only in the initialization of all linear projections (patch embedding, W_Q, W_K, W_V, W_O in each block, FFN layers, classification head). **Xavier** uses Eq. (3). **Orthogonal** applies QR decomposition to a random Gaussian matrix and uses the orthogonal factor; for rectangular shapes, we retain the leading $r \times c$ submatrix with $r \leq c$. We verify $\|W^\top W - I\|_F < 10^{-6}$ at construction. Biases, class tokens, and positional embeddings are initialized identically for both models.

D. Training Protocol

Both models are trained with AdamW (learning rate 10^{-3} , weight decay 10^{-4}), batch size 128, for 15 epochs using cross-entropy loss, with identical random seeds for data shuffling. All computation runs on CPU (AMD Ryzen 5, 16 GB RAM).

E. Diagnostics

During training we log four quantities:

- 1) **Loss and accuracy** — training loss, training accuracy, and test accuracy at each epoch.
- 2) **Per-layer gradient norms** — after each epoch’s final batch, we record $\|\nabla_W \mathcal{L}\|_2$ for every trainable parameter tensor in the model.
- 3) **Jacobian condition numbers** — at epochs 1, 5, 10, and 15, we compute the Jacobian of each transformer block’s output with respect to its input on a single sample using Zygote.jl reverse-mode AD [12], take its singular values via LAPACK, and report $\kappa(J_\ell) = \sigma_{\max}/\sigma_{\min}$.
- 4) **Saliency maps** — after training, for each digit class c , we select one test sample, compute $|\partial \text{logit}_c / \partial X|$ via reverse-mode AD, and average across the embedding dimension of each patch to obtain a 4×4 patch-resolution sensitivity map.

V. RESULTS

We present results in five parts: training dynamics, Jacobian conditioning, gradient norms, saliency, and per-class performance.

A. Training Dynamics

Figure 1 shows training loss and test accuracy across 15 epochs for both models. Orthogonal initialization yields a faster initial descent: at epoch 1, training loss is 0.490 for orthogonal versus 0.591 for Xavier, and orthogonal maintains

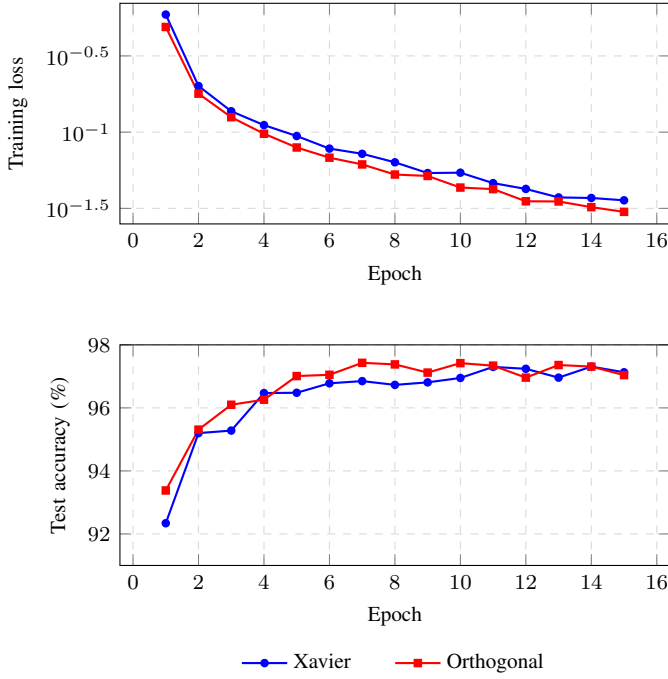


Fig. 1: Training loss (top, log scale) and test accuracy (bottom) over 15 epochs. Orthogonal initialization yields consistently lower loss throughout training.

a lower training loss at every subsequent epoch. Final test accuracies are statistically indistinguishable (97.04% orthogonal vs. 97.13% Xavier) — both models converge to the regime typical of small ViTs on MNIST.

B. Jacobian Condition Numbers

Figure 2 reports per-block Jacobian condition numbers $\kappa(J_\ell)$, measured as described in Section IV, at epochs 1, 5, 10, and 15 (log scale).

At initialization, orthogonal weights yield dramatically better-conditioned Jacobians in both blocks: $\kappa_1^{\text{ortho}} = 2.9 \times 10^6$ vs. $\kappa_1^{\text{xavier}} = 1.3 \times 10^8$ (a 46 $\times$ advantage for block 1), and $\kappa_2^{\text{ortho}} = 911$ vs. $\kappa_2^{\text{xavier}} = 3873$ (a 4.3 $\times$ advantage for block 2). This confirms Prediction 1 of Section III-E: orthogonal initialization places the attention Jacobian dramatically closer to an isometry than Xavier initialization.

Block 1 retains a conditioning advantage for orthogonal initialization throughout training, finishing at $\kappa_1^{\text{ortho}} = 6.2 \times 10^5$ versus $\kappa_1^{\text{xavier}} = 9.0 \times 10^6$ — over an order of magnitude better. Block 2 exhibits more volatile behavior in both models as training progresses; we interpret this volatility in Section ?? in terms of the peaky-attention amplification mechanism identified in Section III-B.

C. Per-Layer Gradient Norms

Figure 3 shows final-epoch gradient norms for each trainable tensor, grouped by functional role. Several patterns are visible.

First, orthogonal initialization produces larger gradient signals in most components, including the patch embedding

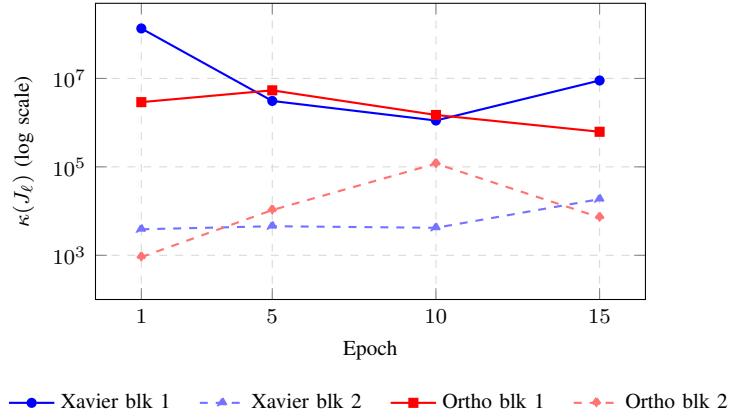


Fig. 2: Jacobian condition number $\kappa(J_\ell)$ for each transformer block over training. Orthogonal initialization starts 46 $\times$ better conditioned in block 1; block 1’s advantage persists by over an order of magnitude throughout training.

projection (0.50 $\rightarrow$ 1.10 for bias), output projections (W_O : 0.35 $\rightarrow$ 0.45), and value projections (W_V : 0.34 $\rightarrow$ 0.41). Larger late-training gradients are generally associated with healthier continued learning, though they do not directly predict final accuracy.

Second, key-projection biases exhibit gradient norms on the order of 10^{-8} in both models. This is not a bug but rather a direct consequence of softmax shift invariance: adding any constant to the key vectors leaves the attention weights unchanged, so $\partial \text{Attn} / \partial b_K = 0$ up to numerical noise. This provides a useful empirical sanity check on our implementation.

Third, deeper blocks exhibit smaller gradient norms than earlier blocks (block 2 norms are uniformly smaller than block 1), reflecting the attenuation expected from a composition of two Jacobians. Figure 3 of the supplementary material shows the full gradient-norm evolution across all epochs as a heat map.

D. Saliency Maps

Figure 4 shows input-sensitivity saliency maps $|\partial \logit_c / \partial X|$ for each digit class c , computed via reverse-mode AD. Both models concentrate sensitivity on the digit strokes, with background regions contributing minimally. The overall patterns are qualitatively similar across the two models, with neither showing obvious artifacts. Differences in spatial focus between the two models are subtle on MNIST; we expect them to become more pronounced on higher-resolution datasets where the network must allocate attention across larger spatial extents.

E. Per-Class Accuracy

Figure 6 presents per-class test accuracy. Overall accuracies are closely matched (Section V-A), but the per-class breakdown reveals complementary inductive biases. Xavier initialization performs noticeably better on digits with primarily straight or vertical structure (1, 4, 5, 7, 9), while orthogonal

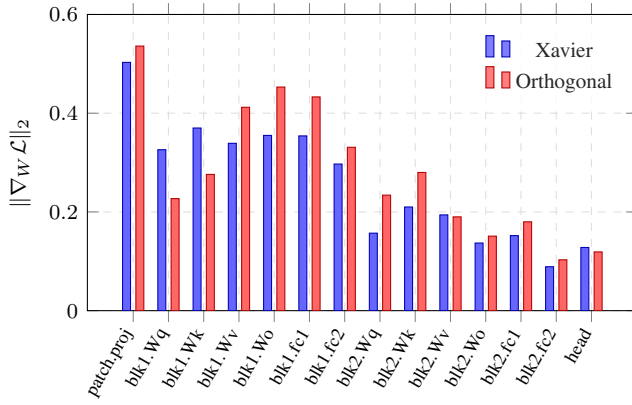


Fig. 3: Final-epoch gradient norms $\|\nabla_W \mathcal{L}\|_2$ for each weight tensor. Orthogonal initialization produces larger late-training gradients in most components, particularly in value and output projections.

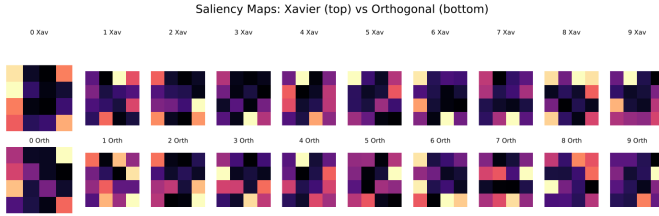


Fig. 4: Per-patch saliency maps $|\partial \logit_c / \partial X|$ aggregated across the embedding dimension, for digits 0–9. Top row: Xavier. Bottom row: Orthogonal. Each cell is a 4×4 grid of patches. Both models concentrate sensitivity on patches covering digit strokes, with qualitatively similar patterns.

initialization performs better on digits containing loops or curves (2, 3, 6, 8). Digit 0 is tied at 98.27%.

F. Gradient Flow Over Training

For completeness, Figure 7 in the supplementary material shows a heat map of per-parameter gradient norms across all 15 training epochs, giving a holistic view of gradient-flow dynamics throughout training.

VI. LIMITATIONS

Several limitations qualify the generality of our findings. First, all experiments are conducted on a single dataset (MNIST) with a single random seed per initialization scheme. Reported condition numbers and gradient norms therefore reflect single-trajectory measurements rather than distributional estimates; a multi-seed study would permit statements about the statistical robustness of the weight-spectrum advantage. Second, the ViT is deliberately small (two blocks, 72,074 parameters), both for CPU tractability and for transparent diagnostics. The interaction between orthogonal initialization and the softmax Jacobian may exhibit qualitatively different behavior in deeper transformers, where the backpropagated Jacobian is a product of many more factors. Third, we measure



Fig. 5: Detail view of patch-level saliency on a single test sample (digit 5). Left: input image. Middle: Xavier saliency. Right: Orthogonal saliency. Both initializations produce structurally similar sensitivity patterns but weight individual patches differently — for instance, the top-left and middle-right patches receive higher sensitivity under Xavier, while the orthogonal map is more evenly distributed across the digit’s spatial extent.

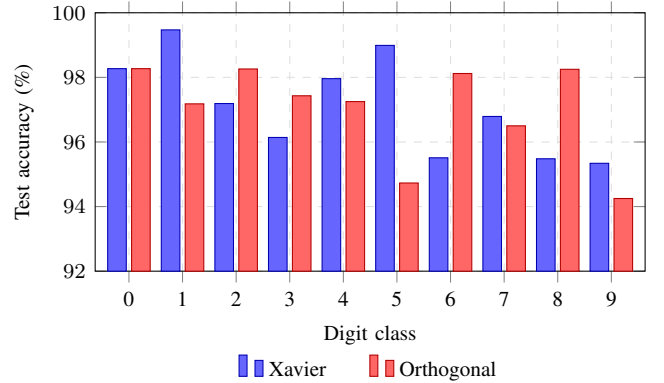


Fig. 6: Per-class test accuracy. Xavier excels on vertically-structured digits (1, 4, 5, 7, 9); orthogonal excels on curved digits (2, 3, 6, 8).

$\kappa(J_\ell)$ on a single input sample per epoch; although the sample is held fixed across models and epochs for a consistent comparison, input-dependence of the Jacobian spectrum is not characterized here. Finally, our saliency-map comparison is qualitative; a quantitative sensitivity metric (for example, pointing-game accuracy or deletion/insertion scores) would be needed to make stronger claims about interpretability differences between the two initialization schemes.

VII. CONCLUSION

We derived the closed-form Jacobians of scaled dot-product attention with respect to its input and its projection weights, and analyzed how the spectra of these Jacobians depend on weight initialization. The derivation identifies two mechanisms controlling Jacobian conditioning: a weight-spectrum contribution through Kronecker factors, and a softmax-Jacobian contribution governed by the sharpness of learned attention distributions. Empirically, on a 72,074-parameter ViT trained on MNIST, orthogonal initialization dramatically improves Jacobian conditioning at initialization relative to Xavier ($46\times$

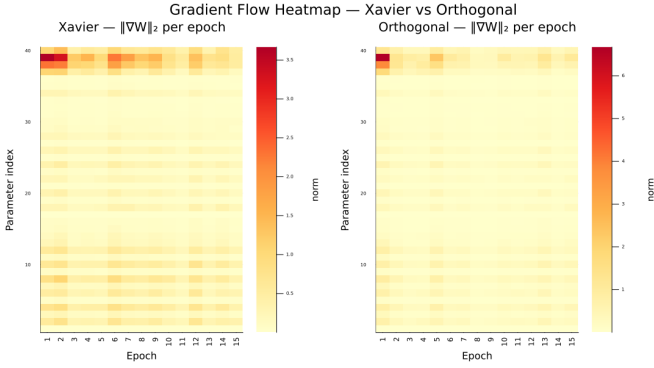


Fig. 7: Full gradient-flow heatmap: rows are trainable parameters (sorted by the architecture’s block order), columns are training epochs, and color encodes $\|\nabla_W \mathcal{L}\|_2$. Left: Xavier. Right: Orthogonal.

and $4.3\times$ lower κ in the two blocks) and yields consistently lower training loss throughout 15 epochs. The initialization-time advantage is preserved by block 1 throughout training, while block 2 exhibits the softmax-dominated regime anticipated by the derivation. Final test accuracies are indistinguishable, but per-class performance reveals complementary inductive biases between the two schemes. Together, these results position matrix calculus not merely as a tool for deriving gradients but as a framework for predicting, through the Jacobian spectra of learned layers, when and why a structural choice like orthogonal initialization helps attention-based architectures.

APPENDIX A

FULL SCORE-CHANNEL AND WEIGHT JACOBIANS

We give the full forms of the Jacobian contributions that were abbreviated in Section III.

A. Score-Channel Contribution to J_X

Let J_σ denote the block-diagonal softmax Jacobian $\text{diag}_i(\text{diag}(A_{i,:}^\top) - A_{i,:}^\top A_{i,:})$. Combining (12), the differential of the attention scores, and the Kronecker identity $\text{vec}(ABC) = (C^\top \otimes A) \text{vec}(B)$, the score channel contributes

$$J_X^{\text{score}} = \frac{1}{\sqrt{d_k}} (V^\top \otimes W_O) J_\sigma (K \otimes W_Q + Q \otimes W_K) \quad (17)$$

acting on $\text{vec}(dX)$, where for compactness we have absorbed permutation matrices that arise from transposes into the Kronecker factor ordering.

B. Jacobians with Respect to W_Q and W_K

Let $M \equiv V^\top \otimes W_O$. Then

$$\frac{\partial \text{vec}(\text{Attn})}{\partial \text{vec}(W_Q)} = \frac{1}{\sqrt{d_k}} M J_\sigma (K^\top \otimes X^\top), \quad (18)$$

$$\frac{\partial \text{vec}(\text{Attn})}{\partial \text{vec}(W_K)} = \frac{1}{\sqrt{d_k}} M J_\sigma^\top (Q^\top \otimes X^\top). \quad (19)$$

Both are linear in J_σ , so peaky attention (small non-null eigenvalues of J_σ) reduces the rank of the effective parameter

Jacobian, consistent with the empirical block-2 dynamics in Section V.

C. Multi-Head Extension

For h heads with per-head dimension $d_k = d/h$, each head contributes an independent block to a block-diagonal Jacobian, and W_O applies uniformly to the concatenated head outputs. The spectral analysis applies head-by-head.

REFERENCES

- [1] X. Glorot and Y. Bengio, “Understanding the difficulty of training deep feedforward neural networks,” in *Proc. 13th Int. Conf. on Artificial Intelligence and Statistics (AISTATS)*, 2010, pp. 249–256.
- [2] K. He, X. Zhang, S. Ren, and J. Sun, “Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification,” in *Proc. IEEE Int. Conf. on Computer Vision (ICCV)*, 2015, pp. 1026–1034.
- [3] A. M. Saxe, J. L. McClelland, and S. Ganguli, “Exact solutions to the nonlinear dynamics of learning in deep linear neural networks,” in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2014.
- [4] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 5998–6008.
- [5] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, “An image is worth 16x16 words: Transformers for image recognition at scale,” in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2021.
- [6] K. B. Petersen and M. S. Pedersen, “The matrix cookbook,” Technical University of Denmark, version 20121115, 2012, available at <https://www2.compute.dtu.dk/pubdb/pubs/3274-full.html>.
- [7] D. Mishkin and J. Matas, “All you need is a good init,” in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2016.
- [8] K. Simonyan, A. Vedaldi, and A. Zisserman, “Deep inside convolutional networks: Visualising image classification models and saliency maps,” in *ICLR Workshop*, 2014.
- [9] M. Innes, “Flux: Elegant machine learning with Julia,” *Journal of Open Source Software*, vol. 3, no. 25, p. 602, 2018.
- [10] R. Xiong, Y. Yang, D. He, K. Zheng, S. Zheng, C. Xing, H. Zhang, Y. Lan, L. Wang, and T.-Y. Liu, “On layer normalization in the transformer architecture,” in *Proc. Int. Conf. on Machine Learning (ICML)*, 2020, pp. 10 524–10 533.
- [11] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [12] M. Innes, A. Edelman, K. Fischer, C. Rackauckas, E. Saba, V. B. Shah, and W. Tebbutt, “A differentiable programming system to bridge machine learning and scientific computing,” 2019.